DP Computer Science: B 4.1.4 Binary Search Trees (Python) (HL Only)

1. Core Concepts & Learning Objectives

Overview

This section provides three detailed, one-hour lessons for Python covering Binary Search Trees (BSTs). Students will explore BST properties, their use in data organisation, and the core algorithms for insertion, deletion, traversal, and searching.

Key Vocabulary

Term	Definition
Tree	A hierarchical data structure consisting of nodes connected by edges
Binary Tree	A tree where each node has at most two children (left and right)
Binary Search Tree (BST)	A binary tree with the property that all values in the left subtree are less than the node's value, and all values in the right subtree are greater
Node	A fundamental building block containing data and references to child nodes
Root	The topmost node of a tree
Leaf	A node with no children
Parent	A node that has one or more child nodes
Child	A node directly connected to another node when moving away from the root
Subtree	A tree consisting of a node and all its descendants
In-order Traversal	Visits left subtree, then node, then right subtree (produces sorted order)
Pre-order Traversal	Visits node, then left subtree, then right subtree
Post-order Traversal	Visits left subtree, then right subtree, then node
Balanced BST	A BST where the heights of left and right subtrees are roughly equal (O(log n) performance)

ATL Skills Focus

Skill	Description
Thinking (Critical Thinking)	Analysing structures to determine if they meet BST criteria; applying BST rules to construct trees

Skill	Description
Thinking (Algorithmic Thinking)	Following and tracing recursive algorithms; decomposing complex processes
Communication	Using subject-specific terminology accurately
Self-Management	Reflecting on outputs; working through complex cases with perseverance

---

2. Lesson 1: Introduction to Binary Search Trees

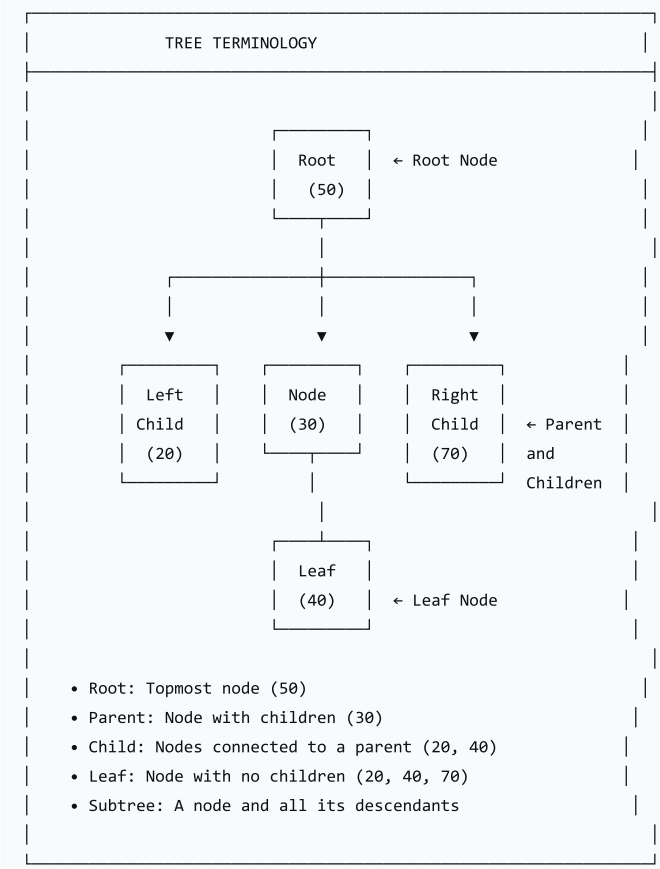
Learning Intentions

By the end of this lesson, students will be able to:

- Define the terms: **tree, binary tree, binary search tree (BST), node, root, leaf, parent, child, and subtree**
- Explain the key properties of a **Binary Search Tree (the BST property)**
- Explain how BSTs are used for efficient data organisation and retrieval
- **Sketch** a BST from a given sequence of numbers

Tree Terminology

text

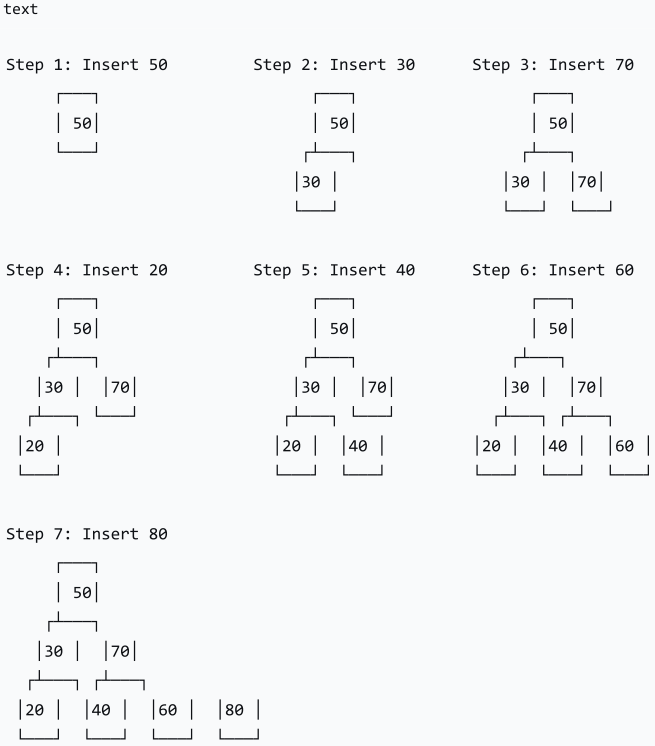


The BST Property

> **For any given node:**
>
> - All values in the **left subtree** are **less than** the node's value
> - All values in the **right subtree** are **greater than** the node's value

Example: Building a BST

Insertion Sequence: 50, 30, 70, 20, 40, 60, 80



Why Use a BST?

Structure	Search Time	Insertion Time	Deletion Time
Unsorted Array	O(n)	O(1) (at end)	O(n)
Sorted Array	O(log n)	O(n)	O(n)
BST (Balanced)	O(log n)	O(log n)	O(log n)
BST (Unbalanced)	O(n)	O(n)	O(n)

---

3. Lesson 2: Traversing and Searching a BST

Learning Intentions

By the end of this lesson, students will be able to:

- Describe the structure of a **Node class** in Python to represent a BST
- Define and trace the three main tree traversal algorithms: **in-order, pre-order, and post-order**
- Explain and trace the algorithm for **searching** for a value in a BST
- Understand the **recursive** nature of these algorithms

Representing a BST in Python

```python
class Node:
    """A node in a Binary Search Tree."""
    def __init__(self, value):
        self.value = value
        self.left = None   # Reference to left child
        self.right = None  # Reference to right child

class BST:
    """A Binary Search Tree."""
    def __init__(self):
        self.root = None   # Reference to the root node
```

Tree Traversals

```text
┌──────────────────────────────────────────────────────────────┐
│             TREE TRAVERSALS                                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│                        ┌─────────┐                           │
│                        │   50    │                           │
│                        └────┬────┘                           │
│                             │                                │
│              ┌──────────────┼──────────────┐                 │
│              │              │              │                 │
│              ▼              ▼              ▼                 │
│         ┌─────────┐    ┌─────────┐    ┌─────────┐           │
│         │   30    │    │   70    │    │         │           │
│         └────┬────┘    └────┬────┘    └─────────┘           │
│              │              │                               │
│         ┌────┴────┐    ┌────┴────┐                          │
│         │ 20 │ 40 │    │ 60 │ 80 │                          │
│         └────┘ └──┘    └──┘ └──┘                          │
│                                                              │
│    In-order:  20, 30, 40, 50, 60, 70, 80  (Sorted!)          │
│    Pre-order: 50, 30, 20, 40, 70, 60, 80                     │
│    Post-order:20, 40, 30, 60, 80, 70, 50                     │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Implementation of Traversals

```python
class BST:
    # ... (previous code)

    def inorder(self, node):
        """In-order traversal: Left → Node → Right"""
        if node is not None:
            self.inorder(node.left)
            print(node.value, end=" ")
            self.inorder(node.right)

    def preorder(self, node):
        """Pre-order traversal: Node → Left → Right"""
        if node is not None:
```

```
            print(node.value, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        """Post-order traversal: Left → Right → Node"""
        if node is not None:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.value, end=" ")
```

Searching a BST

```
python

class BST:
    # ... (previous code)

    def search(self, node, target):
        """Search for a value in the BST."""
        if node is None:
            return False
        if node.value == target:
            return True
        if target < node.value:
            return self.search(node.left, target)
        else:
            return self.search(node.right, target)

    def iterative_search(self, target):
        """Iterative search in the BST."""
        current = self.root
        while current is not None:
            if current.value == target:
                return True
            if target < current.value:
                current = current.left
            else:
                current = current.right
        return False
```

---

4. Lesson 3: Inserting and Deleting Nodes

Learning Intentions

By the end of this lesson, students will be able to:

- Explain and trace the algorithm for **inserting** a new node into a BST
- Explain and trace the algorithm for **deleting** a node from a BST, covering all three cases:
  - Node is a leaf
  - Node has one child
  - Node has two children
- Sketch the state of a BST after a series of **insertions and deletions**

Insertion

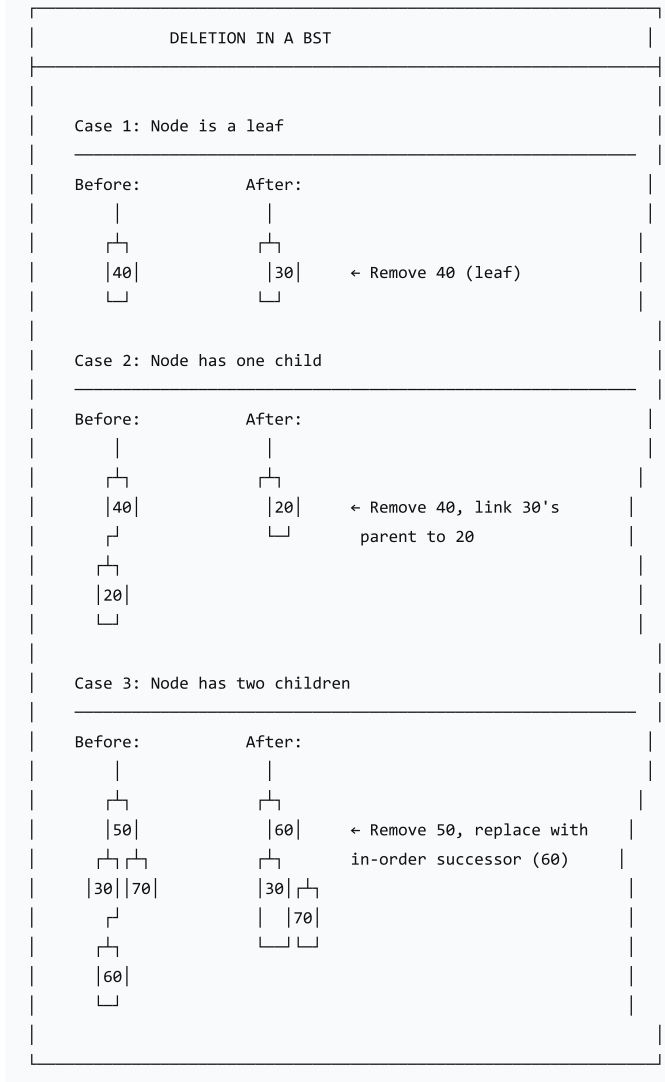
```
python
```

```python
class BST:
    # ... (previous code)

    def insert(self, node, value):
        """Insert a value into the BST."""
        if node is None:
            return Node(value)
        if value < node.value:
            node.left = self.insert(node.left, value)
        else:
            node.right = self.insert(node.right, value)
        return node
```

Deletion: The Three Cases

text



Deletion Implementation

python

```python
class BST:
    # ... (previous code)
```

```
    def delete(self, node, value):
        """Delete a value from the BST."""
        # Base case: value not found
        if node is None:
            return node

        # Search for the node to delete
        if value < node.value:
            node.left = self.delete(node.left, value)
        elif value > node.value:
            node.right = self.delete(node.right, value)
        else:
            # Node found - handle the three cases

            # Case 1: Leaf node
            if node.left is None and node.right is None:
                return None

            # Case 2: One child
            if node.left is None:
                return node.right
            if node.right is None:
                return node.left

            # Case 3: Two children
            # Find the in-order successor (smallest in right subtree)
            successor = self.find_min(node.right)
            # Copy the successor's value to the current node
            node.value = successor.value
            # Delete the successor from the right subtree
            node.right = self.delete(node.right, successor.value)

        return node

    def find_min(self, node):
        """Find the node with the minimum value."""
        current = node
        while current.left is not None:
            current = current.left
        return current
```

---

5. Worksheets

Worksheet	Topic
Worksheet 1	BST terminology, properties, and construction
Worksheet 2	Traversals (in-order, pre-order, post-order) and searching
Worksheet 3	Insertion and deletion operations

---

6. Summary: Key Takeaways

Concept	Key Point
BST Property	Left < Node < Right
In-order Traversal	Produces sorted order

Concept	Key Point
Search Time	O(log n) if balanced
Insertion	Search then insert at the appropriate empty spot
Deletion (Leaf)	Simply remove the node
Deletion (One Child)	Bypass the node; link parent to child
Deletion (Two Children)	Replace with in-order successor

```text
┌─────────────────────────────────────────────────────────┐
│                   KEY REMINDERS                         │
├─────────────────────────────────────────────────────────┤
│                                                         │
│    ✅ BST: Left < Node < Right                          │
│    ✅ In-order traversal = sorted output                │
│    ✅ Search is O(log n) if balanced                    │
│    ✅ Insertion: find location, create node             │
│    ✅ Deletion: 3 cases (leaf, one child, two children) │
│    ✅ Case 3: replace with in-order successor           │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

> *"Binary Search Trees are a powerful data structure that combine the flexibility of linked lists with the efficiency of binary search."*